DAY 03 / 30 SEPTEMBER 2026

From business language to code boundaries

CS403BKK Software Engineering
Harbour.Space Bangkok | Learning notes

We revisited Purchase, Payments and Access, formed teams including Frontend, discussed team accountabilities, began shaping code for extraction and briefly introduced contracts. These notes recap that work; the worked examples help you revisit the ideas.

1. Start with responsibility and meaning

A **domain** is an area of business activity. A **model** selects the concepts and rules needed to reason about it. A **bounded context** marks where a particular model and vocabulary have a consistent meaning. Relationships between contexts must be explicit [1].

Context	Question to bring to its owner
Purchase	What booking is being made, and what price belongs to it?
Payments	What is the outcome of collecting the requested money?
Access	What access is issued for the authorised booking?

The same booking identifier may appear in all three contexts without every context owning the same data or decisions. Frontend is a consumer of application capabilities; a screen can bring several responsibilities together without owning all their rules.

2. A context is not automatically a service

You can first express a boundary through a module inside one application. A separately deployed service adds network calls, independent failures and deployment decisions. Moving a calculation into a module is useful progress, but it does not establish a service or a separate database.

3. Team roles are accountabilities

Product Owner: clarify the user need, priorities and acceptance.
Architect: help the team reason about boundaries, contracts and trade-offs.
SRE: help the team reason about operation, diagnosis and recovery.

Everyone still contributes to development, review and checking. Teams were asked to choose these role holders; the roles are not exclusive implementation jobs.

A SMALL EXTRACTION

Make one rule visible

Yesterday's example: booking-price calculation

4. Separate three different facts

Term	Meaning
Hourly rate	The price per hour used as an input.
Booking price	The monetary amount assigned to a particular booking.
Payment status	Evidence about payment processing; a calculated price alone does not mean money was collected.

For a booking without subscription coverage, the discussed calculation uses the hourly rate at creation and the duration in hours, then rounds to the nearest whole cent, with half-cent ties rounded up [2, 3].

Illustrative arithmetic: 20.00 currency units per hour for 90 minutes gives 30.00 currency units. A calculated 100.5 cents rounds to 101 cents. These numbers illustrate the rule; they are not new business prices.

5. Move the calculation; preserve its behaviour

The application change extracts and names the booking-price helper under Purchase and updates the shared booking caller. The helper calculates an amount. The caller still obtains the rate, handles subscription coverage, validates the booking and records the result [2].

The existing calculation truncates duration to whole seconds before calculating the amount. Preserving that detail is part of a behaviour-preserving extraction. Deciding to change it would be a separate policy change.

Before	After
Booking caller uses a helper among general application code.	Booking caller uses an explicitly named Purchase calculation.
Inputs produce an amount under the existing rule.	The same inputs should produce the same amount.

6. What evidence supports the change?

Use ordinary and rounding-edge examples, then check existing callers. The application PR records 154 passing baseline tests and 160 passing tests after the extraction, including six added calculation examples. It also records an image build and health-handler check [2]. Those are recorded checks for that revision, not proof of every possible behaviour or a production deployment.

Both linked PRs are merged as checked on 1 October. The extraction is still a **module change**. Merged, deployed and useful to a user are different claims.

What can a caller rely on?

A contract makes a boundary usable by someone else.

An interface names the available operations. A useful contract also explains the meaning of inputs and results, the promised behaviour, failure cases and who is responsible. Matching field names alone is not enough.

7. Five questions for any boundary

1. Request: What may the caller ask for?
2. Meaning: What do the values mean, including units and time?
3. Result: What does success establish?
4. Failure: What can be rejected, fail or remain unknown?
5. Ownership: Who decides, and who responds when something changes?

Worked example: a payment request

This is a discussion example, not an agreed or implemented application contract.

Purchase asks Payments to collect an amount for a booking. The teams must agree the currency, amount unit and booking reference. An **accepted** request might only mean processing has started; the contract must say whether money has actually been collected.

If Payments completes the operation but its reply is lost, the caller sees a timeout. That means the outcome is **unknown**, not necessarily failed. A retry is safe only if duplicate handling has been agreed and implemented. A request identifier on its own does not provide that guarantee.

Likewise, successful payment and successful access issuance are separate outcomes. Trace the member journey across the boundaries and ask what each response actually proves.

Check your understanding

Explain these to a teammate using one concrete example:
- Why are booking price and payment status different?
- What stayed the same when the calculation moved?
- What extra work would make this an independently deployed service?
- What may a caller conclude after a timeout?

These prompts support revision and discussion; they are not an additional submission.

References and class examples

[1] Martin Fowler, Bounded Context - martinfowler.com/bliki/BoundedContext.html

[2] Application PR #196: Extract booking price calculation into Purchase
github.com/cs403bkk-2026/spacey/pull/196
Recorded checks: revision bb39ee724046 (30 September 2026).

[3] Business-rules PR #14: Clarify booking price and ownership
github.com/cs403bkk-2026/spacey-business-rules/pull/14
Read alongside the linked student contribution and continuing glossary work.